Introduction to Verilog

Presentation Outline

- Hardware Description Language
- Verilog Module
- Writing a Simple Test Bench

Hardware Description Language

- Describes the hardware of digital systems in a textual form
- Describes the hardware structures and behavior
- Can represent logic diagrams, expressions, and complex circuits
- NOT a software programming language
- Two standard hardware description languages (HDLs)

1. **Verilog** (will be studied in this course)
2. **VHDL**

Verilog = "Verifying Logic"

- Invented as a simulation language in 1984 by Phil Moorby
- Opened to public in 1990 by Cadence Design Systems
- Became an IEEE standard in 1995 (Verilog-95)
- Revised and upgraded in 2001 (Verilog-2001)
- Revised also in 2005 (Verilog-2005)
- Verilog allows designers to describe hardware at different levels
  - Can describe anything from a single gate to a full computer system
- Verilog is supported by the majority of electronic design

Verilog Module

- A digital circuit is described in Verilog as a set of modules
- A module is the design entity in Verilog
- A module is declared using the **module** keyword
- A module is terminated using the **endmodule** keyword
- A module has a name and a list of **input** and **output** ports
- A module is described by a group of statements
- The statements can describe the module structure or

Modeling a Half Adder

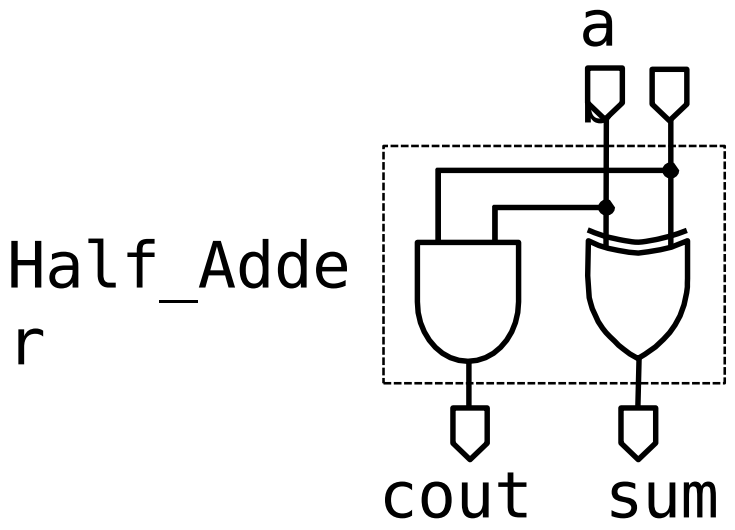
A half adder adds two bits: **a** and **b**

Two output bits:

1. Carry bit: **cout = a · b**
2. Sum bit: **sum = a ⊕ b**

Truth Table

a	b	cout	sum
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



Half_Adder_Verilog_code

```
module half_adder (    input A,
   input B,
   output Sum,
    output Carry
);
  assign Sum = A ^ B;     // XOR for sum
  assign Carry = A & B;   // AND for carry
                           //assign means continuous assignment
                          //The value of Sum is always updated automatically

endmodule
```

Test Bench

- In order to simulate a circuit, it is necessary to apply inputs to the circuit for the simulator to generate an output response
- A test bench is written to **verify the correctness** of a design
- A test bench is written as a **Verilog module with no ports**
- It **instantiates the module** that should be tested
- It **provides inputs** to the module that should be tested
- Test benches can be complex and lengthy, depending on

Half_Adder_Test_Bench

```
`timescale 1ns/1ps 1ps                    //Defines time unit and time precision
module tb_half_adder;
    reg A, B; ;                           //we use reg because we manually assign values
   wire Sum, Carry;                       // Declares outputs as wire

   // Instantiate the Half Adder
   half_adder uut ( .A(A),  .B(B), .Sum(Sum),  .Carry(Carry)  );          //Unit Under Test
                                                                          // Connects testbench signals to module signals
  initial begin
        A = 0; B = 0;              #10;
        A = 0; B = 1;              #10;
        A = 1; B = 0;              #10;
        A = 1; B = 1;              #10;
     $finish; ;                                //Stops the simulation
   end
Endmodule                                     //   Ends the module definition
```

Full Adder

- Full adder adds 3 bits: **a**, **b**, and **c**
- Two output bits:
  1. Carry bit: **cout**
  2. Sum bit: **sum**
- Sum bit is 1 if the number of 1's in the input is odd (odd function)

  sum = (a ⊕ b) ⊕ c
- Carry bit is 1 if the number of 1's in the input is 2 or 3

  cout = a·b + (a ⊕ b)·c

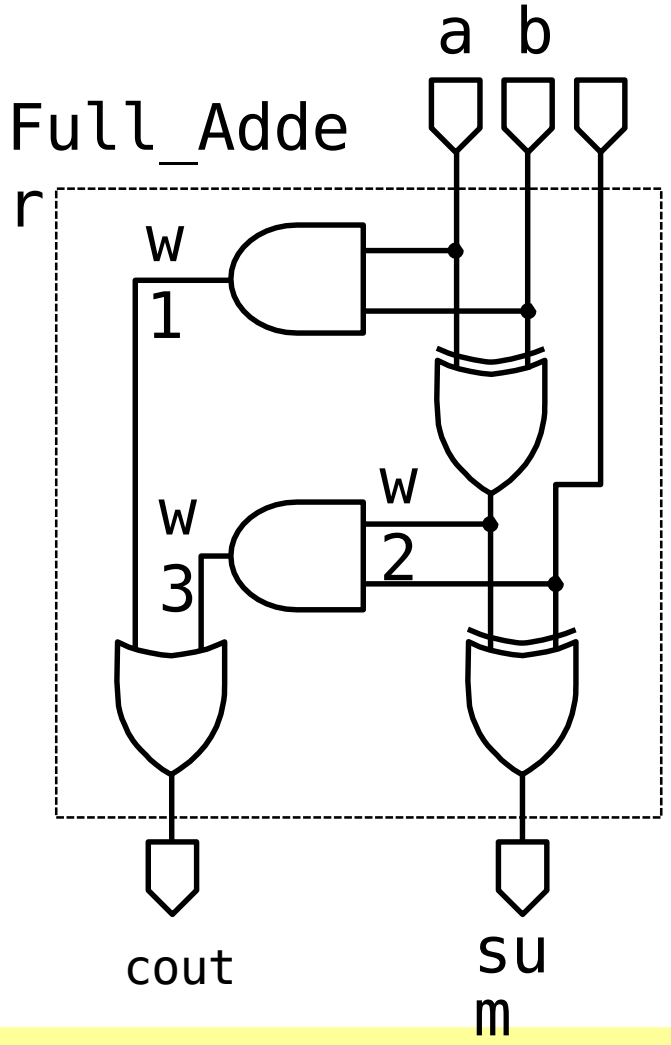
Truth Table

a	b	c	cout	sum
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0

Full Adder Module

```
module Full_Adder(input a, b, c, output cout,
sum);
  wire w1, w2,
  w3;
  and (w1, a,
  b);
  xor (w2, a,
  b);
  and (w3, w2,
  c);
  xor (sum, w2,
  c);

  or  (cout, w1,
  w3);
endmodul
e
```



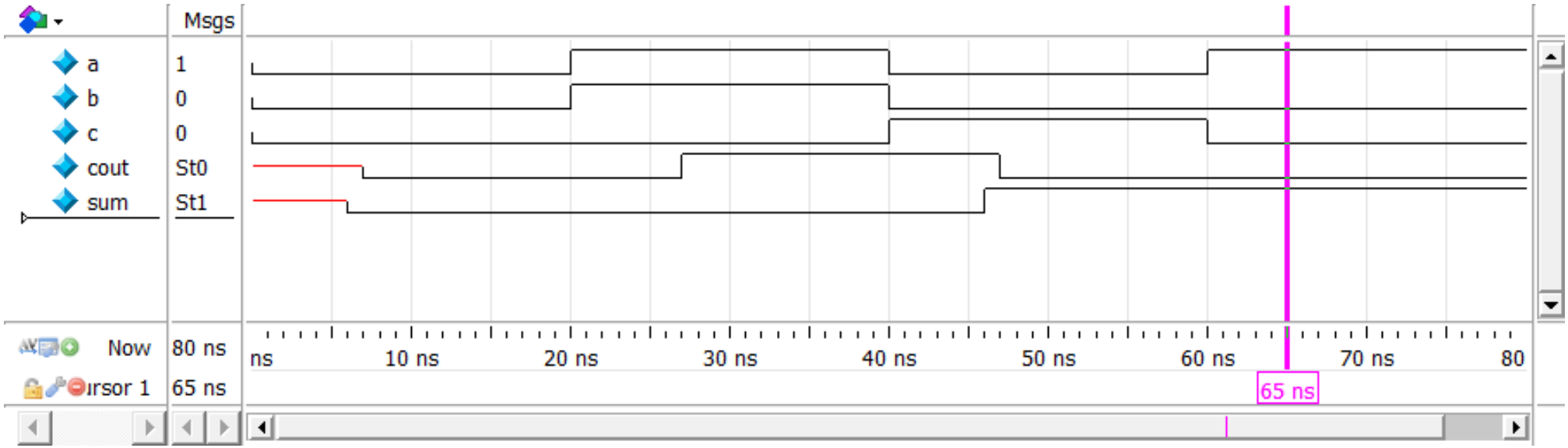
Full_Adder_Verilog_code

```
module full_adder (
    input A,
    input B,
    input Cin,
    output Sum,
    output Cout
   );
    assign Sum  = A ^ B ^ Cin;                          // XOR for sum
   assign Cout = (A & B) | (B & Cin) | (A & Cin);  // Carry logic
                                  //assign means continuous assignment
                              //The value of Sum is always updated automatically
endmodule
```

Full_Adder_Test_Bench

```
`timescale 1ns/1ps                  //Defines time unit and time precision
module tb_full_adder;               //Declares the testbench module
   reg A, B, Cin;                   //we use reg because we manually assign values
   wire Sum, Cout;                  // Declares outputs as wire
   // Instantiate the Full Adder
   full_adder uut (.A(A),.B(B),.Cin(Cin),.Sum(Sum),.Cout(Cout) );    //Unit Under Test
                                                                     // Connects testbench signals to module signals
   initial begin
      A=0; B=0; Cin=0; #10;
      A=0; B=0; Cin=1; #10;
      A=0; B=1; Cin=0; #10;
      A=0; B=1; Cin=1; #10;
      A=1; B=0; Cin=0; #10;
      A=1; B=0; Cin=1; #10;
      A=1; B=1; Cin=0; #10;
      A=1; B=1; Cin=1; #10;
      $finish;                      //Stops the simulation
   end
Endmodule                           //Ends the block and module
```

Running the Simulator



Examine the waveforms to verify the correctness of your design

At $t = 0$ ns, the values of **cout** and **sum** are unknown (shown in red) The **cout** and **sum** signals are delayed by **7ns** and

Continuous Assignment

- The **assign** statement defines continuous assignment
- Syntax: **assign** *name* = *expression*;
- Assigns *expression* value to *name* (output port or wire)
- Examples:

```
assign  x = a&b | c&~d;        // x =  ab + cd'
assign  y =  (a|b) & ~c;       // y =  (a+b)c'
assign  z =  ~(a|b|c);         // z =  (a+b+c)'
assign  sum  = (a^b) ^ c;      // sum  = (a ⊕ b)  ⊕ c
```

Continuous Assignment with Delay

Syntax: **assign** *#delay name = expression*;

The optional *#delay* specifies the delay of the assignment

To have a delay similar to the gate implementation

```
module Full_Adder (input a, b, c, output cout,
sum);
 assign #6 sum = (a^b)^c;
 assign #7 cout = a&b |
  (a^b)&c;
endmodule
```

```
// delay =
6
// delay =
7
```

The order of the **assign** statements does not matter

They are **sensitive** to inputs (a, b, c) that appear in the expressions Any change in value of the input ports (a, b, c) will **re-evaluate** the outputs sum and cout of the **assign** statements

Difference Between **wire** and **reg**

Verilog has two major data types

1. **Net data types:** are connections between parts of a design
2. **Variable data types:** can store data values

- The **wire** is a net data type
  - A wire cannot store a value
  - Its value is determined by its driver, such as a gate, a module output, or continuous assignment
- The **reg** is a variable data type
  - Can store a value from one assignment to the next
  - Used only in procedural blocks, such as the **initial** block

Module Instantiation

- Module declarations are like **templates**
- Module instantiation is like creating an **object**
- Modules are instantiated inside other modules at different levels
- The top-level module does not require instantiation
- Module instantiation defines the **structure** of a digital design
- It produces **module instances** at different levels
- The ports of a module instance must match those declared